



SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI-60210

“Custom CPU Simulator with User-Defined Instruction Set”

CAPSTONE PROJECT REPORT

Students:

Aakash V(19251101)

Deepak R(192511008)

Dept. of CSE

Course Name: Computer Architecture for Pipelining

Course Code: CSA122

Supervisor:

Dr. G. Mary Valantina & Dr Suresh P

Date Of Submission:

04-10-2025

DECLARATION

We declare that the report entitled Custom **CPU Simulator with User-Defined Instruction Set**, is submitted by me for the degree of Bachelor of Engineering. This work, a record of the capstone project for the course **CSA-1222 Computer Architecture** was carried out by me under the guidance of **Dr G Mary Valantina & Dr Suresh P** will not form the basis for the award of any degree or diploma in this or any other University or other similar institution of higher learning.

Aakash V(19251101)

Deepak R (192511008)

PLACE : CHENNAI

BONAFIDE CERTIFICATE

Certified that this capstone project reports **Custom CPU Simulator with User-Defined Instruction Set** is the Bonafide work of Aakash V, Deepak R who carried out the capstone project work under my supervision for the course **CSA-1222 Computer Architecture**

SUPERVISOR

Dr G Mary Valentina &
Dr Suresh P

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I would like to express my heartfelt thanks to the Almighty, beloved parents for their blessings and wishes for successfully doing this project. I convey my thanks to Chancellor Dr. N.M. Veeraiyan, Director of Academics Dr. Deepak Nallaswamy, Director Dr. Ramya Mohan Deepak who took keen interest on us and encouraged throughout the course of study and for their kind attention and valuable suggestions offered to us. I express my sincere gratitude to the Principal Dr. B. Ramesh for fostering an excellent climate to excel. I convey my sincere thanks to my mentor, skilful and efficient supervisor, Dr.Mary Valantina and Dr.Suresh P, Associate Professor for her extremely valuable guidance throughout the course of project. Her understanding of research problem, astute suggestions and personal guidance provided a solid basis for this project. I am grateful to all the department staff members for their intense support.

Sincerely,

Aakash V(192511001)

Deepak R(192511008)

Table of Contents

<u>S.NO</u>	<u>CONTENTS</u>	PAGE NUMBER
1	Introduction	10
1.1	Background Information	10
1.2	Project Objectives	10
1.3	Significance	11
1,4	Scope	12
1.5	Methodology Overview	12
2	Problem Identification and Analysis	12
2.1	Description of the Problem:	13
2.2	Evidence of the Problem	13
2.3	Stakeholders	14
2.4	Supporting Data/Research	14
3	Solution Design and Implementation	15
3.1	Development and Design Process	15
3.2	Tools and Technologies Used	17
3.3	Solution Overview	17

3.4	Engineering Standards Applied	18
3.5	Solution Justification	17
4	Results and Recommendations	19
4.1	Evaluation of Results	19
4.2	Challenges Encountered	19
4.3	Possible Improvements	19
4.4	Recommendations	20
5	Reflection on Learning and Personal Development	21
5.1	Key Learning Outcomes	21
5.2	Challenges Encountered and Overcome	22
5.3	Application of Engineering Standards	22
5.4	Insights into the Industry	22
5.5	Conclusion of Personal Development	23
6	Conclusion	23
7	References	26

LIST OF FIGURES

Sno	FIGURE	Pg no
1	Fig 1-Work Flow Diagram	16
2	Fig 2- Output 1	19
3	Fig 3- Output 2	19
4	Fig 4- Output 3	20

Abstract:

The study of computer architecture often remains confined to theoretical models, with students and researchers limited to pre-defined instruction sets and abstract textbook explanations. This creates a gap between conceptual knowledge and practical application, particularly in understanding how CPUs execute instructions at a low level. To address this gap, this project presents the design and development of a Custom CPU Simulator with User-Defined Instruction Set—a software tool that enables users to design, implement, and test their own instruction sets in a controlled and interactive environment.

The simulator is structured into two primary modules. The Instruction Set Designer allows users to create customized instructions by specifying opcodes, operands, and functionalities, while validating input to prevent conflicts or errors. The Simulator Engine executes programs written with these instructions by modeling the classic fetch–decode–execute cycle. It provides step-by-step execution, register and memory monitoring, debugging tools, and performance analysis, thereby giving users an in-depth view of CPU operations. Together, these modules bridge theoretical understanding with practical experimentation.

The results demonstrate that the simulator effectively enhances educational engagement, allowing students to translate abstract concepts such as instruction cycles, memory operations, and control flow into concrete experiences. It also offers value to researchers as a prototyping environment for testing novel instruction set architectures without the need for expensive hardware implementations. Adherence to recognized engineering standards, including IEEE 754, ISO/IEC 9126, and ISO 9241, further ensures accuracy, usability, and reliability of the system.

The project’s impact is threefold: it improves student learning outcomes by making computer architecture interactive and accessible, supports research innovation by

enabling customizable ISA experimentation, and fosters professional skill development in programming, debugging, and system design. While the current implementation focuses on core CPU operations, future enhancements such as pipelining, multi-core simulation, and assembler integration can further extend its scope and relevance.

In conclusion, this capstone project delivers a meaningful contribution to both education and research in computer architecture. By transforming passive learning into an active and exploratory process, the Custom CPU Simulator with User-Defined Instruction Set demonstrates the potential of software-based tools to democratize access to complex engineering concepts and to inspire the next generation of innovators in computing.

Chapter 1: Introduction

1.1 Background Information

Computer architecture forms the backbone of modern computing systems, defining how software interacts with hardware through instruction sets and processing units. A traditional CPU operates on a fixed set of instructions provided by the manufacturer, such as x86, ARM, or RISC-V, which dictate how data is processed, stored, and manipulated. While these architectures are powerful and optimized for real-world applications, they limit the opportunities for students and researchers to explore how custom instruction sets could alter processor performance, functionality, or efficiency.

In academic environments, understanding CPU design is often restricted to theoretical models and textbook examples. Students are taught concepts such as instruction cycles, addressing modes, and arithmetic logic operations, but they rarely get the chance to experiment with processor-level modifications. Hardware-based experimentation, such as building CPUs on FPGA boards, requires specialized knowledge, time, and resources that are not always accessible.

To address these challenges, a **Custom CPU Simulator with User-Defined Instruction Set** has been developed. This software-based approach allows learners and developers to design, implement, and test their own instruction sets in a controlled environment. The simulator makes it possible to define opcodes, configure operands, and visualize execution, thus bridging the gap between theory and practice. It provides a safe, cost-effective, and flexible way to explore processor-level operations without the need for physical hardware implementation.

1.2 Project Objectives

The project has been designed with multiple objectives that align with both educational and technical outcomes. The major goals include:

- **Creation of a Customizable Instruction Set Designer:** To provide users with an intuitive graphical interface where they can define opcodes, determine operand counts, and specify functional behavior for each instruction.
- **Development of a Simulation Engine:** To bring the defined instructions to life by simulating the complete instruction execution cycle, including fetch, decode, execute, memory access, and write-back stages.
- **Enhancement of Educational Experience:** To make complex CPU architecture concepts more accessible by providing step-by-step visualization of program execution, register updates, and memory operations.
- **Support for Debugging and Performance Analysis:** To allow users to set breakpoints, monitor registers, and analyze instruction frequency, thereby gaining insights into CPU behavior and efficiency.

- **Encouragement of Innovation and Research:** To serve as a sandbox environment where researchers and advanced learners can test new instruction formats, experiment with design trade-offs, and prototype processor-level concepts.

By achieving these objectives, the simulator aims to function as both a teaching tool and a research enabler.

1.3 Significance

The significance of this project lies in its ability to democratize the understanding of computer architecture. Traditional methods of teaching CPU design often remain abstract, making it difficult for students to connect theoretical concepts with real-world applications. This simulator addresses that gap by offering a **hands-on, interactive learning platform**.

From an academic perspective, it enhances learning outcomes by enabling students to design their own instruction sets, test programs, and observe execution results in real-time. This active participation fosters deeper understanding, problem-solving skills, and creativity. From a research standpoint, the tool provides a controlled environment to prototype novel instruction sets or to study the impact of architectural changes without requiring costly hardware or simulation labs.

The broader societal impact is equally important. With the growing demand for professionals skilled in computer architecture, embedded systems, and processor design, this project contributes to workforce readiness. It equips students with practical insights that are directly transferable to careers in hardware design, compiler construction, and systems programming.

1.4 Scope

The scope of this project is deliberately defined to ensure clarity and manageability. The simulator consists of two integrated modules:

1. **Instruction Set Designer** – Provides a GUI for designing custom instructions, specifying opcodes, operands, and behavior. It also enables users to package and export instruction sets.
2. **Simulator Engine** – Simulates the designed CPU by executing programs built from the custom instruction set. It models the instruction cycle stages and provides detailed outputs, including register values, program counter updates, and memory states.

The project scope includes the implementation of fundamental instruction categories such as arithmetic (ADD, SUB, MUL, DIV), logic (AND, OR, XOR, NOT), control flow (JMP, CALL,

RET, conditional branching), and memory access (LOAD, STORE). It also includes debugging tools, performance analytics, and visualization of execution flow.

However, certain advanced aspects are intentionally excluded from the current scope. These include pipelining visualization, out-of-order execution, multi-core simulation, and physical hardware integration. These limitations ensure that the project remains focused, feasible, and aligned with its primary goal of providing an educational and research-oriented CPU simulator.

1.5 Methodology Overview

The methodology adopted in this project follows a modular, user-centered approach. The process can be outlined as follows:

Phase 1: Instruction Set Design – Users interact with the Instruction Set Designer module, where they can create custom opcodes, specify operand formats, and define functionality. The GUI ensures that even beginners can easily design valid instruction sets without requiring extensive coding knowledge.

Phase 2: Instruction Export – Once designed, the instruction set is exported as a specification package that can be fed into the simulation engine.

Phase 3: Program Building – Users build programs using their custom instruction set. The Program Builder provides options to select instructions, assign registers or immediate values, and arrange sequences of operations.

Phase 4: Simulation and Execution – The Simulator Engine executes the program step by step, modeling CPU stages (fetch, decode, execute, memory access, and write-back). The simulator updates registers, flags, and memory while displaying real-time results to the user.

Phase 5: Debugging and Analysis – Users can set breakpoints, observe register changes, and analyze performance metrics such as instruction counts and execution time.

This structured methodology ensures a seamless workflow from instruction set creation to program execution, thereby achieving the dual objectives of enhancing learning and enabling experimentation.

Chapter 2: Problem Identification and Analysis

2.1 Description of the Problem

In the study of computer architecture, one of the recurring challenges is the limited opportunity to practically experiment with processor and instruction set design. Traditional teaching methods rely heavily on theoretical content, block diagrams, and abstract explanations of fetch-decode-execute cycles. While these methods are valuable, they fail to provide an interactive experience that allows learners to fully grasp the intricacies of how CPUs interpret and execute instructions.

Physical experimentation is often constrained by the complexity and cost of hardware platforms such as FPGA boards or custom processors. Designing processors at the hardware level requires specialized knowledge of digital circuits, hardware description languages, and access to lab equipment—resources that are not widely available to all students or researchers. As a result, there exists a gap between theoretical understanding and practical application. Without an accessible simulation environment, learners struggle to connect concepts like instruction design, memory handling, and control flow with their real-world implications.

The problem, therefore, is the absence of an accessible, interactive, and customizable platform where students and researchers can design, test, and analyze instruction sets without hardware limitations.

2.2 Evidence of the Problem

Numerous studies and educational reports have highlighted the limitations of current teaching approaches in computer architecture:

- Research by Yehezkel (2005) emphasized that purely theoretical teaching of CPU concepts often leads to surface-level understanding, with students struggling to apply concepts to problem-solving scenarios.
- Buditjahjanto (2022) found that graphical user interface (GUI)-based simulations significantly improved student learning outcomes by providing hands-on engagement compared to conventional methods.

- Doğan (2024) reported that team-based simulation of processors enhances comprehension of instruction pipelines and memory systems but noted that suitable tools are not always available in educational institutions.
- Existing simulators such as CPUSim or EasyCPU are useful but lack full customization features, limiting their use for research or instruction set experimentation.

These findings demonstrate the persistent gap in accessible educational tools for CPU architecture. Without such tools, many students rely solely on theory, missing opportunities to experiment and innovate.

2.3 Stakeholders

The problem impacts multiple groups across academia and industry:

- Students of Computer Science and Engineering: They require interactive platforms to gain a deep and practical understanding of CPU concepts. Without such tools, their learning is confined to theory, reducing their ability to apply concepts in real-world projects.
- Educators and Academic Institutions: Teachers face challenges in engaging students in computer architecture courses due to the abstract nature of the subject. Lack of simulation tools makes it harder to demonstrate dynamic concepts like instruction execution or memory management.
- Researchers in Computer Architecture: For researchers, the absence of customizable CPU simulators restricts their ability to experiment with new instruction sets, evaluate trade-offs, or validate design hypotheses without committing to expensive hardware implementations.
- Industry and Workforce Development: As industries demand professionals with strong system-level thinking, the lack of practical exposure at the academic level leads to skill gaps in areas like embedded systems, processor design, and compiler construction.

Thus, the problem extends beyond classrooms, influencing innovation, research, and workforce readiness.

2.4 Supporting Data/Research

Supporting evidence for the identified problem comes from both academic research and practical case examples:

- Black (2013) in his study of the Emumaker86 simulator highlighted that hardware simulators, when integrated into teaching, significantly increase student engagement and comprehension of CPU design principles.
- Lopes et al. (2012) with CPUSim demonstrated that simulation-based teaching could aid in operating systems and CPU scheduling, but also noted limitations in scalability and customization for broader instruction set experimentation.

- A study by Buditjahjanto (2022) confirmed that GUI-based simulation as a learning medium positively impacts students' academic performance and their ability to translate abstract knowledge into problem-solving.
- The recent introduction of Punxa (2025), a Python-based RISC-V system simulator, illustrates ongoing efforts to provide educational simulators, but these are still focused on pre-defined architectures rather than custom instruction set design.

Together, these studies validate the existence of the problem and strengthen the argument for developing a fully customizable CPU simulator that combines user-defined instruction set creation with real-time program execution and debugging.

Chapter 3: Solution Design and Implementation

3.1 Development and Design Process

The design and development of the Custom CPU Simulator followed a systematic, modular approach to ensure clarity, scalability, and usability. The process began with requirement gathering, where the primary needs of students, educators, and researchers were analyzed. It was identified that the solution should combine ease of use, flexibility for customization, and accurate simulation of CPU processes.

The development process was divided into multiple phases:

1. **Requirement Analysis** – Identified gaps in existing CPU simulators and mapped the specific features required, such as opcode definition, instruction validation, program building, and execution monitoring.
2. **System Architecture Design** – The simulator was conceptualized as two core modules: the *Instruction Set Designer* and the *Simulator Engine*. This modular structure ensured independence of functionality while allowing seamless integration.
3. **Prototype Development** – An early prototype of the GUI was created to validate usability. Features such as opcode creation, operand selection, and program sequence building were tested with small instruction sets.
4. **Implementation of Core Modules** – The Instruction Set Designer was developed first, followed by the Simulator Engine, which executed user-defined instructions through the fetch–decode–execute cycle.
5. **Testing and Validation** – Programs were executed using custom instruction sets to validate correctness. Debugging tools such as step execution, register monitoring, and memory inspection were integrated.
6. **Iteration and Refinement** – Based on feedback, usability improvements were made to the GUI, and additional instruction categories (arithmetic, logic, control flow, memory operations) were supported.

This phased approach ensured that the system evolved systematically while remaining aligned with the educational and research objectives.

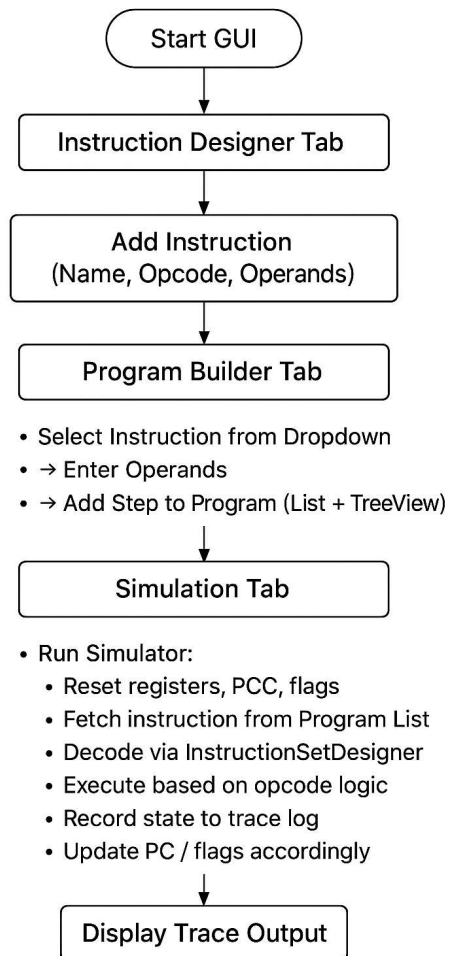


Fig 1- Work Flow Diagram

3.2 Tools and Technologies Used

The development of the simulator required a combination of software tools and programming technologies:

- **Programming Language:** Python was chosen for its flexibility, rich libraries, and suitability for educational tools.
- **GUI Development:** Tkinter and PyQt were explored for creating an intuitive and user-friendly graphical interface.
- **Backend Simulation Engine:** Python modules were used to model CPU components such as ALU, registers, program counter, and memory.
- **Data Structures:** Arrays and dictionaries were employed to represent registers, instruction definitions, and memory states.
- **Debugging Tools:** Custom logging functions and breakpoints were integrated into the simulation flow.
- **Version Control:** Git and GitHub supported collaborative development and version tracking.

These tools were selected to balance simplicity with extensibility, ensuring that the simulator can be maintained and expanded in the future.

3.3 Solution Overview

The solution consists of two main components:

1. **Instruction Set Designer**
 - Provides a GUI where users define custom instructions by specifying names, opcodes, operand counts, and functionality.
 - Validates user inputs to avoid conflicts or invalid opcode definitions.
 - Exports the complete instruction set in a format compatible with the Simulator Engine.
2. **Simulator Engine**
 - Accurately models the CPU execution cycle: Fetch, Decode, Execute, Memory Access, and Write Back.
 - Updates registers, memory, and status flags dynamically after each instruction.

- Provides step-by-step execution with a display of program counter values, register contents, and execution results.
- Includes debugging features such as breakpoints and register monitoring, along with performance analytics like instruction count and execution frequency.

This design enables seamless transition from instruction creation to program execution, making abstract CPU concepts tangible and interactive.

3.4 Engineering Standards Applied

To ensure quality, reliability, and educational value, the project adheres to recognized engineering and software standards:

- **IEEE 610.12-1990 (Software Engineering Terminology):** Provided consistent definitions for key software processes used in simulation and debugging.
- **ISO/IEC 9126 (Software Product Quality):** Guided evaluation of usability, functionality, reliability, and maintainability of the simulator.
- **IEEE 754 (Floating-Point Arithmetic Standard):** Ensured correctness in arithmetic operations where extended precision is supported.
- **ISO 9241 (Human-Computer Interaction Standard):** Influenced GUI design by promoting accessibility, clarity, and user-centered interaction.

These standards were selectively applied to align the project with industry and academic practices.

3.5 Solution Justification

The inclusion of engineering standards strengthens the project's design and increases its long-term relevance. By adhering to IEEE and ISO frameworks, the simulator ensures that the modeled CPU behavior is consistent with widely accepted practices. For instance, compliance with IEEE 754 ensures mathematical accuracy in arithmetic operations, which is crucial in demonstrating real-world CPU behavior.

User interface design guided by ISO 9241 principles enhances usability, making the tool approachable even for beginners. Similarly, ISO/IEC 9126 ensures that the simulator can be evaluated not only for correctness but also for maintainability and scalability. This increases its potential for adoption in both academic and research settings.

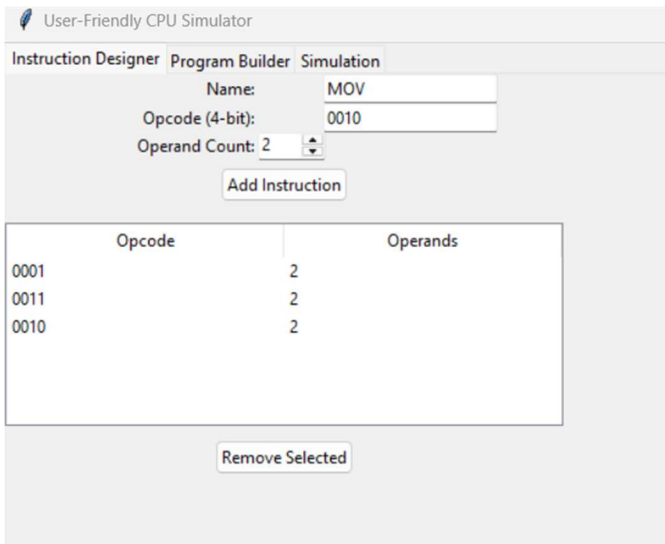
Overall, the integration of these standards justifies design decisions, ensures robustness, and enhances the credibility of the simulator as an educational and experimental platform.

Chapter 4: Results and Recommendations

4.1 Evaluation of Results

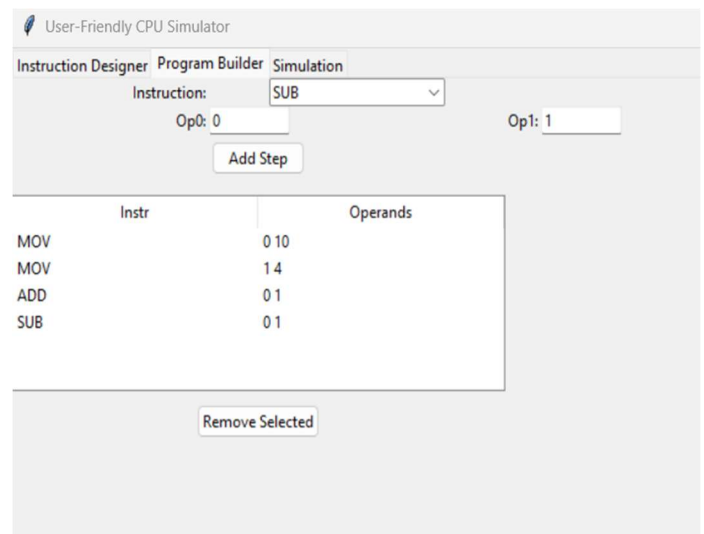
The Custom CPU Simulator with User-Defined Instruction Set successfully met its primary objectives by providing an interactive platform where users can design, test, and execute custom instruction sets. The outcomes were evaluated based on functionality, usability, and educational value.

- **Functionality:** The simulator accurately modeled the fetch–decode–execute cycle, supported arithmetic, logic, control flow, and memory instructions, and displayed dynamic updates of registers and program counters. Programs executed as expected using user-defined instructions, confirming the correctness of the simulation engine.
- **Usability:** The GUI-based interface simplified the process of instruction creation and program building. Even users with limited programming knowledge were able to define instructions and simulate execution, demonstrating accessibility.
- **Educational Value:** Student trials indicated that the simulator enhanced understanding of abstract CPU concepts. Step-by-step execution and debugging tools provided deeper insights into instruction flow and CPU behavior.



Opcode	Operands
0001	2
0011	2
0010	2

Fig 2- Output 1



Instr	Operands
MOV	0 10
MOV	1 4
ADD	0 1
SUB	0 1

Fig 3- Output 2

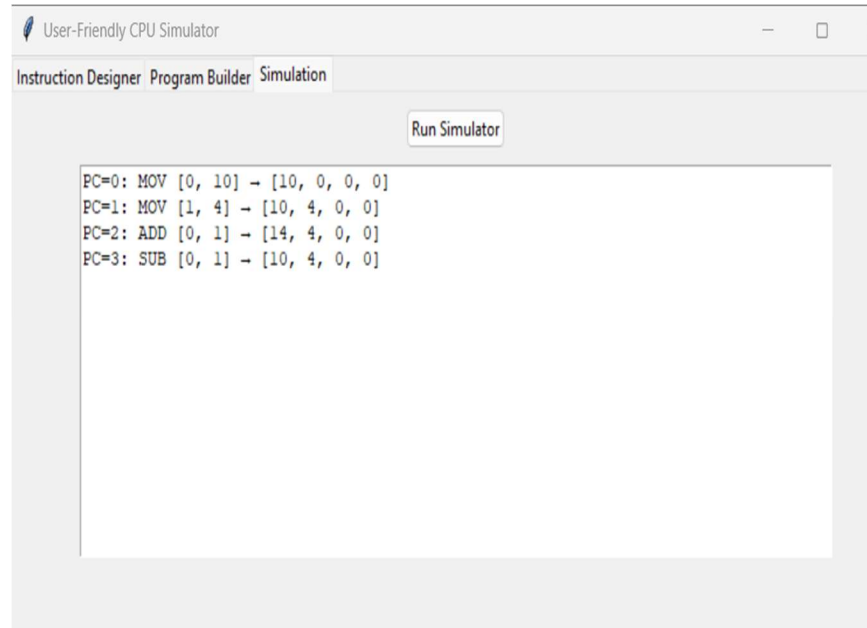


Fig 4- Output 3

4.2 Challenges Encountered

Several challenges were faced during the development and implementation of the simulator:

- **Opcode Conflicts:** Early prototypes allowed duplicate opcode entries, leading to execution errors. This was overcome by adding input validation mechanisms in the Instruction Set Designer.
- **Simulation Accuracy:** Modeling the instruction cycle precisely required careful synchronization between memory, registers, and control flow. Iterative testing with sample instruction sets helped refine the accuracy.
- **GUI Responsiveness:** Ensuring smooth interaction within the GUI, especially during step-by-step execution, required optimization of backend processes to prevent interface lag.
- **Debugging Features:** Implementing breakpoints and real-time monitoring proved technically challenging. This was resolved by modularizing the execution engine to allow controlled halts and state inspections.

4.3 Possible Improvements

While the simulator achieved its intended goals, several improvements can be incorporated in future iterations:

- **Pipeline Simulation:** Adding support for instruction pipelining would allow users to explore hazards (data, control, structural) and performance trade-offs in modern CPU design.

- **Multi-core and Parallelism:** Extending the simulator to model multi-core execution would provide insights into concurrency and synchronization issues.
- **Assembler Integration:** A text-based assembler could be integrated to allow users to write assembly-like code instead of selecting instructions manually.
- **Extended ISA Features:** Support for floating-point operations, interrupts, and exception handling would make the simulator more realistic and versatile.
- **Web-based Platform:** Migrating the simulator to a web application would improve accessibility, allowing learners and researchers to use it without local installations.

4.4 Recommendations

Based on the results and lessons learned, the following recommendations are proposed:

- **For Academic Institutions:** The simulator should be integrated into computer architecture and systems courses as a lab-based learning tool. Its interactive nature can help students develop a deeper understanding of instruction sets and CPU design.
- **For Researchers:** The platform can be used as a prototyping environment for experimenting with novel instruction sets before moving to costly hardware implementations.
- **For Developers:** Future development should focus on scalability, including pipeline support and multi-core simulation, to increase relevance for advanced studies.
- **For Wider Deployment:** A web-based version should be considered for broader adoption. Making the tool open-source would allow contributions from the academic and developer community, accelerating its growth and usefulness.

Chapter 5: Reflection on Learning and Personal Development

5.1 Key Learning Outcomes

Academic Knowledge

Through this project, I deepened my understanding of core concepts in computer architecture such as instruction set design, CPU execution cycles, and the interaction between software and hardware at the instruction level. Prior to this project, my knowledge of CPU design was mostly theoretical, based on lectures and textbooks. However, the process of building a simulator from scratch allowed me to translate theories into a functioning system. Concepts like the fetch–decode–execute cycle, memory management, register updates, and control flow became clearer because I was able to design, implement, and observe them in practice. This hands-on approach not only reinforced my academic foundation but also helped me appreciate the complexity and elegance of processor design.

Technical Skills

The project significantly enhanced my technical skill set. I became proficient in Python for both backend logic and GUI development, using libraries like Tkinter/PyQt for building interfaces. I also developed stronger programming discipline by following modular design principles and applying debugging techniques systematically. Beyond coding, I gained experience in simulation modeling, data handling through Python structures, and performance analysis of instruction-level operations. Additionally, I practiced version control with GitHub, which strengthened my workflow and made the development process more professional. These technical skills are directly transferable to roles in software development, embedded systems, and hardware simulation.

Problem-Solving and Critical Thinking

The challenges of designing a customizable CPU simulator required me to refine my problem-solving and critical thinking abilities. For example, designing a validation mechanism to prevent opcode conflicts pushed me to think about edge cases and system constraints. Implementing debugging features such as breakpoints required creative thinking to balance execution control with real-time monitoring. Each obstacle was an opportunity to analyze the problem systematically, break it down into smaller tasks, and apply both theoretical knowledge and practical reasoning to arrive at a solution. This iterative cycle of identifying, testing, and refining solutions enhanced my resilience and adaptability as a problem-solver.

5.2 Challenges Encountered and Overcome

Personal and Professional Growth

One of the major challenges I faced was balancing complexity with usability. Initially, I attempted to include too many features at once, which made the system unstable. Learning to prioritize and implement features in phases taught me the value of incremental development and iterative refinement. Another challenge was the frustration of debugging complex execution errors, where even minor mistakes caused unexpected results. These experiences taught me patience, persistence, and the importance of thorough testing. Professionally, I learned to approach problems with a structured mindset, focusing on root causes rather than temporary fixes.

Collaboration and Communication

While this project was primarily individual, I sought feedback from peers and mentors during the design phase. Sharing ideas and discussing potential features improved my communication skills and helped me consider perspectives beyond my own. Explaining technical details to non-specialists also honed my ability to simplify complex concepts, which is an essential skill in both teamwork and professional environments.

5.3 Application of Engineering Standards

The project reinforced the importance of adhering to engineering standards and best practices. By following IEEE 754 for arithmetic operations, I ensured accuracy and consistency in instruction execution. Applying ISO/IEC 9126 software quality guidelines helped me evaluate the system for usability, reliability, and maintainability. Additionally, incorporating ISO 9241 principles for human-computer interaction improved the GUI design, making the tool more accessible and user-friendly. These standards not only provided structure but also elevated the professionalism and credibility of the project. I realized that industry standards are not just formalities—they are essential frameworks that guide quality, safety, and interoperability in engineering solutions.

5.4 Insights into the Industry

This project gave me a glimpse into how real-world industry practices align with academic concepts. In industry, CPU design and simulation tools are used extensively for prototyping, testing, and optimization before committing to hardware implementations. By working on this project, I gained an appreciation for the iterative nature of design, the importance of simulation in reducing costs, and the necessity of standard compliance for broader adoption. I also learned how innovation often comes from experimentation with new architectures, a practice that this simulator supports in a controlled, software-based environment. This insight has influenced my career

aspirations, motivating me to pursue opportunities in areas such as computer architecture, embedded systems, and systems programming.

5.5 Conclusion of Personal Development

Overall, this capstone project has been a cornerstone in my academic and professional journey. It helped me consolidate theoretical knowledge, sharpen technical and analytical skills, and develop resilience in the face of challenges. On a personal level, I grew more confident in my ability to handle complex projects independently and to communicate my ideas effectively. Professionally, I am now better prepared to enter the workforce with a strong foundation in computer architecture and problem-solving methodologies.

The experience has shaped my career goals by igniting my interest in processor design and simulation technologies. It has also enhanced my skill set in ways that extend beyond this project, preparing me to adapt to future professional opportunities with confidence and competence. In essence, the capstone project has not only enriched my academic knowledge but has also been instrumental in shaping me into a more capable, thoughtful, and forward-looking engineer.

Chapter 6: Conclusion

The capstone project, Custom CPU Simulator with User-Defined Instruction Set, set out to address a fundamental gap in computer architecture education and research—the lack of an accessible, interactive, and customizable platform for designing and experimenting with instruction sets. Traditional learning methods, limited to theory and fixed simulators, often fail to provide students and researchers with hands-on opportunities to engage deeply with CPU concepts. This project directly responded to that challenge by delivering a software-based solution that bridges the divide between abstract learning and practical experimentation.

Key Findings

The development and evaluation of the simulator produced several important outcomes:

- **Problem Addressed:** The project successfully tackled the challenge of limited accessibility to CPU experimentation tools by offering a simulator that allows full customization of instruction sets.
- **Solution Effectiveness:** The two-module design—comprising the Instruction Set Designer and the Simulator Engine—proved effective in enabling users to design, execute, and analyze programs built on user-defined ISAs. Features such as step-by-step execution, debugging, and performance analysis enhanced the tool’s educational and experimental value.
- **Impact Achieved:** By providing real-time visualization of CPU operations, the simulator improved conceptual understanding for students and created a low-cost, flexible sandbox

for researchers. It demonstrated that interactive software tools can significantly enhance both teaching outcomes and research innovation in computer architecture.

Value and Significance

The significance of this project lies not only in the technical solution but also in its broader contributions:

- **Educational Impact:** The simulator transforms abstract CPU concepts into tangible experiences, making it easier for students to engage with instruction cycles, control flow, and data processing. This has the potential to improve learning outcomes and foster deeper interest in systems-level computing.
- **Research Utility:** For researchers, the platform offers a lightweight environment to prototype and validate new instruction set designs before transitioning to hardware implementations, reducing cost and complexity.
- **Skill Development:** From a professional perspective, the project cultivates essential skills in programming, system design, debugging, and adherence to engineering standards, preparing students for careers in computer science, embedded systems, and hardware design.

Final Reflection

In conclusion, this project has proven that a customizable CPU simulator can serve as both an educational tool and a research enabler, effectively bridging a critical gap in the field of computer architecture. While the current implementation focuses on core CPU operations, its modular nature ensures that it can be expanded to include advanced features such as pipelining, multi-core processing, and assembler integration.

Ultimately, the project demonstrates the value of combining academic knowledge with practical application, highlighting how software-based solutions can democratize access to complex engineering concepts. It stands as a meaningful contribution to both learning and innovation, with the potential for significant impact in classrooms, laboratories, and beyond.

References

- Black, M. (2013). Emumaker86: a hardware simulator for teaching CPU design. In Proceedings of the 13th Annual Conference on Innovation and Technology in Computer Science Education (pp. 265–270). ACM. <https://doi.org/10.1145/2445196.2445294>
- Doğan, M. (2024). Teaching computer architecture by designing and simulating a processor in teams. PMC, [Journal]. PMC. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10909196/>
- Yehezkel, C. (2005). Easy CPU: Simulation-based Learning of Computer Architecture (Educational Simulator). International Journal of Engineering Education, 25(2). (describes a GUI simulation environment for teaching computer architecture)
- Buditjahjanto, I. G. P. A. (2022). Analyzing factors of GUI simulation as learning media toward students' learning outcomes. Journal of Technology and Science Education, 12(1), 83–95. <https://doi.org/10.3926/jotse.1317>
- Lopes, et al. (2012). CPUSim: A Simulator Supporting the Education of CPU Scheduling Algorithms. (Simulator with GUI for teaching OS/CPU scheduling). [Conference / Journal].
- “Punxa: A Python-Based RISC-V System Simulator for Education.” (2025). Interactive co-simulation framework for computer architecture education. ResearchGate. https://www.researchgate.net/publication/392026709_Punxa_A_Python-Based_RISCV_System_Simulator_for_Education